JAVA COLLECTIONS & STRINGS

50 Questions with Simple Answers

Beginner-friendly notes covering the Java Collection Framework, List, Set, Queue, Map, Iterator, and String methods.

Prepared for study and exam revision

A. Java Collection Framework

1. What is the Java Collection Framework?

Answer: The Java Collection Framework is a set of interfaces and classes used to store and manage groups of objects. Examples include ArrayList, LinkedList, HashSet, TreeSet, and HashMap.

2. Why do we use collections instead of arrays?

Answer: Arrays have a fixed size. Many collections can grow or shrink dynamically and provide useful methods for adding, removing, searching, and processing elements.

3. What is the difference between Collection and Collections?

Answer: Collection is an interface that represents a group of objects. Collections is a utility class that provides static methods such as sort(), reverse(), and min().

4. What is the difference between Collection and Map?

Answer: Collection stores individual elements. Map stores key-value pairs. Map is part of the Collections Framework but does not extend the Collection interface.

5. What is the difference between List, Set, and Queue?

Answer: List stores ordered elements and allows duplicates. Set does not allow duplicate elements. Queue holds elements for processing, often in FIFO order, though some queues use other ordering rules.

6. What is the difference between ArrayList and LinkedList?

Answer: ArrayList uses a resizable array and is generally fast for accessing elements by index. LinkedList uses linked nodes and can be useful for frequent insertions or removals through an iterator or at the ends.

7. What are the important features of ArrayList?

Answer: ArrayList is resizable, maintains insertion order, allows duplicates and null values, and provides index-based access. It is not synchronized by default.

8. How do you add elements to an ArrayList?

Answer: Use add(element). Example: ArrayList<String> names = new ArrayList<>(); names.add("Ravi");

9. What is the difference between add() and add(index, element)?

Answer: add(element) appends an element to the end of a List. add(index, element) inserts the element at the specified index and shifts later elements to the right.

10. How do you access an element from an ArrayList?

Answer: Use get(index). Example: names.get(0) returns the first element. List indexes start at 0.

11. How do you remove an element from an ArrayList?

Answer: Use remove(index) to remove the element at an index, or remove(object) to remove the first matching element.

12. What is the difference between remove(int index) and remove(Object)?

Answer: remove(int index) removes the element at that position. remove(Object) removes the first element equal to the given object. With an ArrayList<Integer>, use Integer.valueOf(5) to clearly remove the value 5 rather than index 5.

13. How do you find the size of an ArrayList?

Answer: Use size(). Example: names.size() returns the number of elements currently in the list.

14. How do you check whether an ArrayList contains a particular element?

Answer: Use contains(element). It returns true if the list contains an element equal to the specified value; otherwise it returns false.

15. How do you iterate through an ArrayList?

Answer: You can use a for loop with indexes, an enhanced for loop, an Iterator, or forEach(). Example: for (String name : names) { System.out.println(name); }

16. What is a Set?

Answer: A Set is a collection that does not allow duplicate elements, according to its equality rules.

17. What is the main difference between List and Set?

Answer: A List allows duplicates and supports positional access by index. A Set prevents duplicates and does not provide index-based access.

18. Does a HashSet allow duplicate elements?

Answer: No. Adding an element that is already present does not create another entry. HashSet allows at most one null element.

19. Does a HashSet maintain insertion order?

Answer: No. HashSet does not guarantee insertion order or sorted order.

20. What is the difference between HashSet and LinkedHashSet?

Answer: HashSet does not guarantee iteration order. LinkedHashSet maintains insertion order while still preventing duplicates.

21. What is the difference between HashSet and TreeSet?

Answer: HashSet has no guaranteed order and generally offers fast basic operations. TreeSet keeps elements sorted according to their natural ordering or a supplied Comparator.

22. What is the purpose of TreeSet?

Answer: TreeSet stores unique elements in sorted order and is useful when you need to keep values ordered automatically.

23. How does TreeSet maintain its elements?

Answer: TreeSet uses a tree-based structure and orders elements by natural ordering or a Comparator. Elements considered equal by that ordering are treated as duplicates.

24. What is a Queue?

Answer: A Queue is a collection designed to hold elements before processing. Many queues follow first-in, first-out (FIFO) order.

25. What is the difference between add() and offer() in a Queue?

Answer: Both try to insert an element. add() may throw an exception if insertion fails; offer() returns false when insertion cannot be completed, such as in a capacity-restricted queue.

26. What is the difference between poll() and remove() in a Queue?

Answer: Both remove and return the head element. poll() returns null if the queue is empty, while remove() throws an exception.

27. What is the difference between peek() and element()?

Answer: Both inspect the head without removing it. peek() returns null if the queue is empty, while element() throws an exception.

28. What is a Map in Java?

Answer: A Map stores key-value pairs. Each key is unique, and a key can be used to find its associated value.

29. What is the difference between a Map and a Set?

Answer: A Set stores unique elements. A Map stores key-value pairs with unique keys. A Map's keySet() view is a Set of its keys.

30. Does a HashMap allow duplicate keys?

Answer: No. A key can appear only once. If you put a value using an existing key, the old value is replaced.

31. Can a HashMap contain duplicate values?

Answer: Yes. Different keys can map to the same value. HashMap also permits one null key and multiple null values.

32. What is the difference between HashMap and LinkedHashMap?

Answer: HashMap does not guarantee iteration order. LinkedHashMap normally maintains insertion order, or can be configured to maintain access order.

33. What is the difference between HashMap and TreeMap?

Answer: HashMap does not guarantee key order. TreeMap sorts keys by their natural ordering or a supplied Comparator.

34. What is the purpose of keySet(), values(), and entrySet()?

Answer: keySet() returns a Set view of all keys. values() returns a Collection view of values. entrySet() returns a Set view of key-value entries.

35. What is an Iterator, and why is it used with collections?

Answer: An Iterator lets you visit collection elements one by one using hasNext() and next(). Its remove() method can safely remove the current element during iteration when supported.

B. Strings in Java

36. What is a String in Java?

Answer: String is a Java class used to represent a sequence of characters. String objects are immutable.

37. Why is String immutable in Java?

Answer: Once created, a String's contents cannot change. This makes strings safer to share, supports the String pool, and helps provide predictable behavior.

38. What is the difference between String and StringBuilder?

Answer: String is immutable, so changes create a new String. StringBuilder is mutable and is more efficient for many repeated modifications in a single thread.

39. What is the difference between StringBuilder and StringBuffer?

Answer: Both are mutable character sequences. StringBuilder is generally faster but not synchronized; StringBuffer synchronizes its methods for thread safety.

40. What is the difference between == and equals() when comparing Strings?

Answer: == checks whether two references point to the same String object. equals() checks whether their character contents are equal.

41. What is the String pool?

Answer: The String pool is a special area used to reuse String instances with the same interned value. String literals are normally pooled, helping reduce duplicate String objects.

42. What is the use of the length() method?

Answer: length() returns the number of UTF-16 code units in a String. Example: "Hello".length() returns 5.

43. What is the use of the charAt() method?

Answer: charAt(index) returns the char at the specified zero-based index. An invalid index causes StringIndexOutOfBoundsException.

44. What is the use of the substring() method?

Answer: substring() extracts part of a String. The starting index is included and the ending index is excluded. Example: "Computer".substring(0, 3) returns "Com".

45. What is the difference between equals() and equalsIgnoreCase()?

Answer: equals() compares String contents with case sensitivity. equalsIgnoreCase() compares contents without distinguishing uppercase and lowercase letters.

46. What is the use of contains()?

Answer: contains() checks whether a String includes a specified character sequence and returns true or false. Example: "Hello Java".contains("Java") returns true.

47. What is the use of indexOf() and lastIndexOf()?

Answer: indexOf() returns the index of the first occurrence of a character or substring. lastIndexOf() returns the index of its last occurrence. Both return -1 if it is not found.

48. What is the difference between replace() and replaceAll()?

Answer: replace() replaces literal character or character-sequence matches. replaceAll() uses a regular expression (regex) to identify what to replace.

49. What is the use of trim()?

Answer: trim() removes leading and trailing characters whose code points are at most U+0020. It does not remove whitespace from the middle of a String.

50. How would you reverse a String in Java?

Answer: A simple way is to use StringBuilder: String reversed = new StringBuilder("Java").reverse().toString(); The result is "avaJ".